

COS30018 - Option C - Task 2 Report

Data processing 1

Student Name: Kha Anh Nguyen

Student ID: 103814796

Date: August 26, 2025

Course: COS30018 - Intelligent Systems

Project: FinTech101 Stock Price Prediction System

Github repo: [Leissha/stock_prediction](https://github.com/Leissha/stock_prediction)

Table of Contents

COS30018 - Option C - Task 2 Report	1
1. Code structure	1
2. Function to load and process dataset	2
3. Appendix.....	6

1. Code structure

For better code handling, I chose to write code in OOP, modularized and packaged different functions with their main functionality.

- **data_preprocessing/**

data_processor.py: main pipeline (for data_processing), it calls:

- a. data_loading.py: yfinance download + local cache
- b. data_splitting.py: options for ratio/date/random splits
- c. handle_nans.py: NaN detection and interpolation
- d. create_sequence.py: LSTM sliding window creation

- **train.py:** call data_processor, orchestrates training, prediction, metrics, plots
- **utils:** helpers for file handling, evaluation and plotting.

2. Function to load and process dataset

Implemented in dev/data_preprocessing/data_processor.py: data_processing():

a. Allow date flexibility

This function helps fetch data with the given date time.

Sample snippet (+pseudo code):

```
def load_stock_data(
    company,
    start_date,
    end_date,
    cache_dir='dev/cache/raw_data'):
    # Compose a cache filename
    file_path = os.path.join(cache_dir, "raw_data",
    f"{company}_{start_date}_to_{end_date}.pkl")

    # Check if cached data existss
    Check if file existst:
    else:
        logger.info(f"Downloading data for {company}")
        data = yf.download(tickers=company, start=start_date, end=end_date)

        if data is not None and not data.empty:
            # yfinance may return a MultiIndex; flatten to a single level
            if isinstance(data.columns, pd.MultiIndex):
                data.columns = data.columns.get_level_values(0)

            # Save to cache
            save_data(data, file_path)
        else:
            raise ValueError(f"Failed to download data for {company}")
    return data
```

b. Handle NaNs

handle_nans(df) detects missing values and applies linear interpolation on numeric columns with logging

Sample snippet:

```
def handle_nans(df):
    """
    Handle NaN values using linear interpolation
    """
    if df.isnull().any().any():
        # Only interpolate numerical columns to avoid issues with date/string columns
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        df[numeric_cols] = df[numeric_cols].interpolate(
            method='linear', # Linear interpolation between known values
            limit_direction='both' # Fill gaps in both forward and backward
            directions
        )
        logger.info(f"Interpolated NaN values in columns: {numeric_cols.tolist()}")
    else:
        logger.info("No NaN values detected")
    return df
```

c. Split data flexibility

We can choose splitting methods using args.parser:

1. date/ratio: chronological split at index $\text{int}(\text{len}(\text{df}) * (1 - \text{test_size}))$
- random: `train_test_split(..., shuffle=True)`, not good for time series

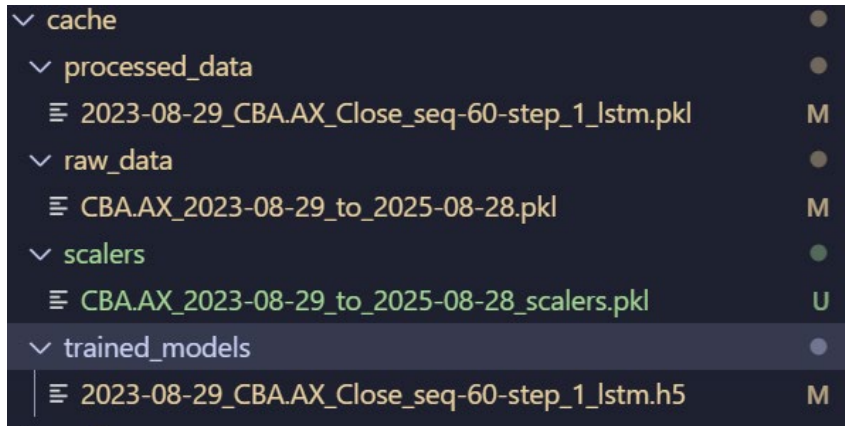
Sample snippet:

```
def split_data(df, method='date', test_size=0.2, random_state=42):
    """
    Split data using different methods
    - 'date'/ratio': Chronological split, respects temporal order (recommended
    for time series)
    - 'random': Random sampling (breaks temporal dependencies, use with caution)
    """
    if method in ['date', 'ratio']:
        split_point = int(len(df) * (1 - test_size))
        train_df = df.iloc[:split_point].copy()
        test_df = df.iloc[split_point:].copy()

    elif method == 'random':
        train_df, test_df = train_test_split(
            df,
            test_size=test_size,
            random_state=random_state,
            shuffle=True
        )
```

d. Cache data

I cached past data in cache/ folder, and checked if the files exist before commencing the next steps:



Sample code for check cached raw data before downloading from Yahoo Finance.

```
# Compose a cache filename
file_path = os.path.join(cache_dir, "raw_data",
f"{company}_{start_date}_to_{end_date}.pkl")

# Check if cached data exists
if os.path.exists(file_path):
    logger.info(f"Loading cached data for {company}")
    data = pd.read_pickle(file_path)
    # Also normalize cached data columns if needed
    if isinstance(data.columns, pd.MultiIndex):
        data.columns = data.columns.get_level_values(0)
    return data
else:
    logger.info(f"Downloading data for {company}")
    data = yf.download(tickers=company, start=start_date, end=end_date)
```

e. Cache feature scalers

- Since each feature has different values range, I separated MinMaxScaler for each feature.
- And fit on train dataset only to avoid data leakage.
- Scalers also stored in data class for quick test.

```

if scale:

    train_scaled = np.zeros_like(train_df[feature_columns].values)
    test_scaled = np.zeros_like(test_df[feature_columns].values)

    for i, feature in enumerate(feature_columns):
        # Create and store individual scaler for each feature
        scaler_key = f"{ticker}_{feature}"
        self.scalers[scaler_key] = MinMaxScaler()

        # Fit scaler only on training data to prevent data leakage
        train_scaled[:, i] = self.scalers[scaler_key].fit_transform(
            train_df[[feature]].values
        ).reshape(-1)

        test_scaled[:, i] = self.scalers[scaler_key].transform(
            test_df[[feature]].values
        ).reshape(-1)

    # Cache all feature scalers for future inference use
    scaler_bundle = {
        "scalers": self.scalers,                # per-feature scalers dict
        "feature_columns": feature_columns,      # column order
        "target_scaler_key": f"{ticker}_{target_feature}", # just the key string
    }
    save_data(scaler_bundle, os.path.join(self.cache_dir, "scalers",
        f"{ticker}_{start_date}_to_{end_date}_scalers.pkl"))

```

- To predict, we can simply inverse-transform target features if the dataset is scaled.

```

if target_scaler is not None:
    actual_prices = target_scaler.inverse_transform(
        y_test.reshape(-1, 1)
    ).reshape(-1)

    predicted_prices = target_scaler.inverse_transform(
        predictions.reshape(-1, 1)
    ).reshape(-1)

else:

```

```
# No scaling was applied, use raw values
```

All functions above will be packaged into a module and called at **train.py** for model training and evaluation:

```
# Create DataProcessor instance
processor = DataProcessor(cache_dir='dev/cache')

# Use the comprehensive data processing method
data = processor.data_processing(
    start_date=START_DATE,
    end_date=END_DATE,
    ticker=TICKER,                    # Stock company
    lag_days=LAG_DAYS,               # learning window size
    lookup_step=1,                   # prediction horizon
    shuffle=SHUFFLE,                 # shuffle training set or not
    splitting_method=SPLIT_METHOD,   # ratio /date /random split
    test_size=TEST_SIZE,
    target_feature='Close',          # Predict Close price
    scale=SCALE
)
```

3. Appendix

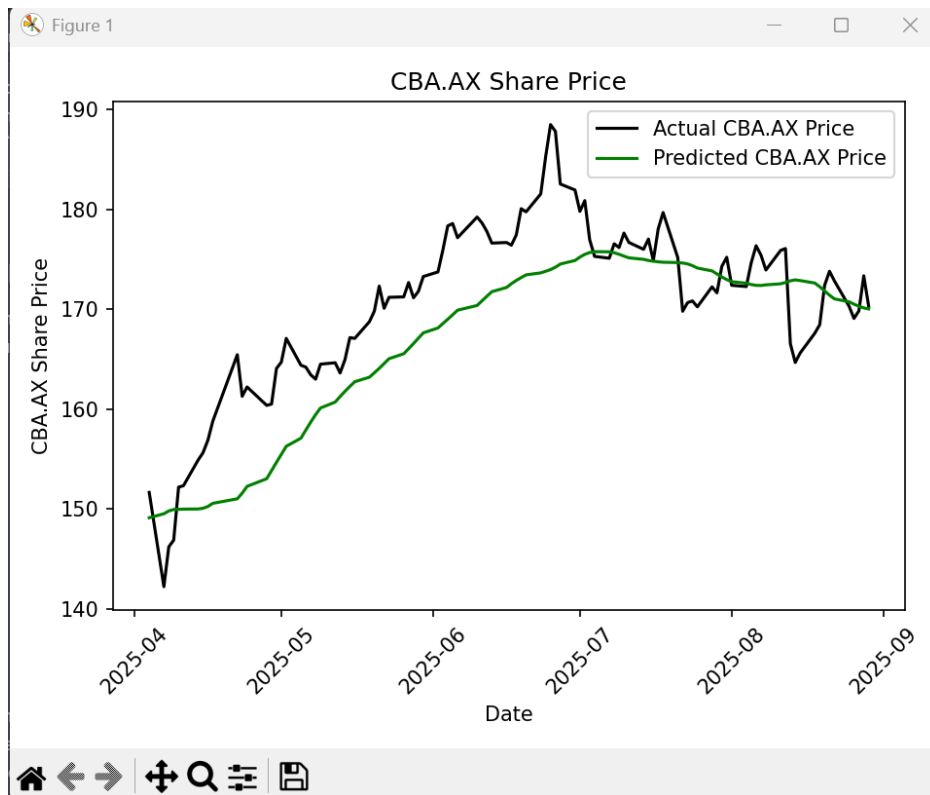
1. First run, data will be cached:

```
(venv) PS C:\Users\User\OneDrive - Swinburne University\IS\assignment1> python dev/train.py
2025-08-28 21:57:23.327067: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-28 21:57:26.996695: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
=====
STOCK PREDICTION PIPELINE
=====
2025-08-28 21:57:28.773 | INFO | __main__:train:85 - === TRAINING PHASE ===
2025-08-28 21:57:28.774 | INFO | data_preprocessing.data_loading:load_stock_data:28 - Downloading data for CBA.AX
C:\Users\User\OneDrive - Swinburne University\IS\assignment1\dev\data_preprocessing\data_loading.py:21: FutureWarning: YF.download() has changed argument auto_adjust default to True
data = yf.download(tickers=company, start=start_date, end=end_date)
[*****100%*****] 1 of 1 completed
2025-08-28 21:57:29.876 | INFO | utils.file_handling:save_data:34 - Data cached to dev/cache/raw_data/CBA.AX_2023-08-29_to_2025-08-28.pkl
2025-08-28 21:57:29.876 | INFO | data_preprocessing.data_processor:data_processing:85 - Loaded data: (506, 5) from 2023-08-29 to 2025-08-28
2025-08-28 21:57:29.876 | INFO | data_preprocessing.data_processor:data_processing:86 - Available columns: ['Close', 'High', 'Low', 'Open', 'Volume']
2025-08-28 21:57:29.877 | INFO | data_preprocessing.data_processor:data_processing:102 - Input features (OHLCV): ['close', 'high', 'low', 'open', 'volume']
2025-08-28 21:57:29.877 | INFO | data_preprocessing.data_processor:data_processing:103 - Target feature: close
2025-08-28 21:57:29.877 | INFO | data_preprocessing.handle_nans:handle_nans:28 - No NaN values detected
2025-08-28 21:57:29.878 | INFO | data_preprocessing.data_splitting:split_data:22 - Splitting data using method: date, test_size: 0.2
2025-08-28 21:57:29.878 | INFO | data_preprocessing.data_splitting:split_data:30 - Chronological split at index 404
2025-08-28 21:57:29.878 | INFO | data_preprocessing.data_processor:data_processing:110 - Data split - Train: (404, 5), Test: (102, 5)
2025-08-28 21:57:29.881 | INFO | data_preprocessing.data_processor:data_processing:137 - Feature 'close' scaled: range 90.0856 to 161.9139
2025-08-28 21:57:29.882 | INFO | data_preprocessing.data_processor:data_processing:137 - Feature 'high' scaled: range 90.6146 to 163.0793
2025-08-28 21:57:29.883 | INFO | data_preprocessing.data_processor:data_processing:137 - Feature 'low' scaled: range 90.0201 to 160.2435
2025-08-28 21:57:29.885 | INFO | data_preprocessing.data_processor:data_processing:137 - Feature 'open' scaled: range 90.5537 to 163.0793
2025-08-28 21:57:29.886 | INFO | data_preprocessing.data_processor:data_processing:137 - Feature 'volume' scaled: range 207240.0000 to 7340421.0000
2025-08-28 21:57:29.886 | INFO | data_preprocessing.data_processor:data_processing:139 - Applied separate MinMaxScaler to 5 features
2025-08-28 21:57:29.886 | INFO | data_preprocessing.data_processor:data_processing:146 - Creating sequences with lag_days=60, lookup_step=1
2025-08-28 21:57:29.886 | INFO | data_preprocessing.create_sequence:create_sequences:28 - Target indices: [0] for columns: ['close']
2025-08-28 21:57:29.888 | INFO | data_preprocessing.create_sequence:create_sequences:45 - Created 344 sequences with lag_days=60, lookup_step=1
2025-08-28 21:57:29.888 | INFO | data_preprocessing.create_sequence:create_sequences:28 - Target indices: [0] for columns: ['close']
2025-08-28 21:57:29.888 | INFO | data_preprocessing.create_sequence:create_sequences:45 - Created 102 sequences with lag_days=60, lookup_step=1
2025-08-28 21:57:29.889 | INFO | utils.file_handling:save_data:34 - Data cached to dev/cache/scalers/CBA.AX_scalers.pkl
2025-08-28 21:57:29.889 | INFO | data_preprocessing.data_processor:data_processing:205 - =====
2025-08-28 21:57:29.889 | INFO | data_preprocessing.data_processor:data_processing:206 - DATA PROCESSING COMPLETE
2025-08-28 21:57:29.889 | INFO | data_preprocessing.data_processor:data_processing:207 - Training sequences: X(344, 60, 5) -> y(344, 1)
2025-08-28 21:57:29.889 | INFO | data_preprocessing.data_processor:data_processing:208 - Test sequences: X(102, 60, 5) -> y(102, 1)
```

2. Second run, load cached data (if exists):

```
(venv) PS C:\Users\User\OneDrive - Swinburne University\IS\assignment1> python dev/train.py
2025-08-28 22:01:37.445535: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-28 22:01:39.495393: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-28 22:01:39.495393: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-28 22:01:39.495393: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-08-28 22:01:40.372271: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
2025-08-28 22:01:40.478 | INFO | __main__:train:146 - Using existing trained model
2025-08-28 22:01:40.478 | INFO | __main__:train:149 - === TESTING PHASE ===
4/4 ----- 0s 78ms/step
2025-08-28 22:01:41.350 | INFO | __main__:predict_and_transform:71 - Inverse transformed using scaler: CBA.AX_close
Trading metrics saved to: dev/results/2023-08-29_CBA.AX_Close_seq-60-step_1_lstm.csv
2025-08-28 22:01:43.446 | INFO | __main__:train:190 - Test results saved to: dev/results/2023-08-29_CBA.AX_Close_seq-60-step_1_lstm.csv
=====
PIPELINE COMPLETED SUCCESSFULLY
=====
```

3. V0.1 prediction:



4. Evaluation:

```
TEST DATASET PERFORMANCE METRICS
=====
Latest predicted price: $170.03
Model loss on test data: 0.006552600301802158
Trading accuracy (profitable trades): 0.4608
Total buy profit: $21.07
Total sell profit: $7.49
Total profit: $28.56
Profit per trade: $0.2800
Profitable trades: 47/102 test samples
Trading threshold: 0.5%

MODEL CONFIGURATION
=====
Training features: ['close', 'high', 'low', 'open', 'volume']
Target feature: close
Feature scaling applied: True
```